Serverless Journal Archive

AWS Architecture Portfolio Final
CS79c Compute Engines for AWS

By: Nina Janev
Repository: https://github.com/yanevah/cs79cFinal

Executive Summary

A historical archive of PDF journals (one journal per month, published in two parts, filed in a folder per year from 1967 onward) had no practical and inexpensive way for users to browse or download it, and maintaining a server was not desirable for what is a small, bursty workload. I built a serverless portal where a visitor picks a year and downloads Part1 or Part 2 of any month. It is for a small group of readers and one administrator who adds PDFs.

Essentially, A static website on S3 is delivered through CloudFront, which also fronts an API Gateway HTTP API. Lambda functions read a DynamoDB catalog and hand out five-minute pre-signed URLs, so the PDFs stay in a private S3 bucket and download straight from S3. A third Lambda, triggered by S3 upload events, indexes each new PDF automatically, and the entire stack is one CloudFormation template that deploys and deletes cleanly.

Scale of the deployed archive: Scanning is still ongoing, approximately 500 journals across 48 years. Region: US West (Oregon), us-west-2. Evidence screenshots (running site, a working download, the CloudFormation stack, DynamoDB items, S3 bucket, Lambda functions and the indexer log) are in the repository's `evidence/` folder.

Architecture

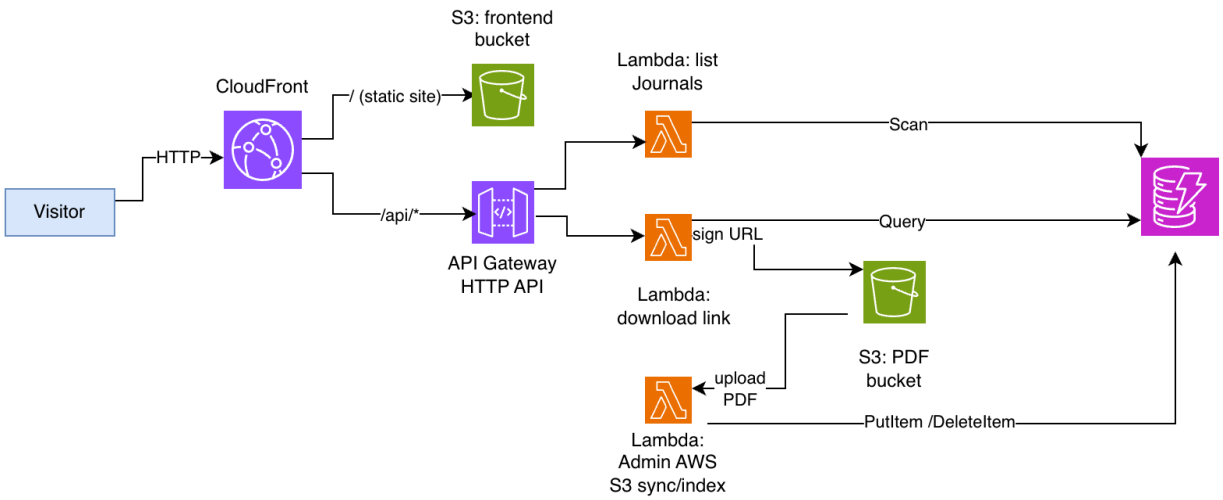


Figure 1.

How a request flows.

- **Browse**. The browser loads the site from CloudFront (S3 origin), then calls `/api/journals`. A Lambda scans DynamoDB and returns year, month, and part.
- **Download**. A click calls `/api/download?year=&month=&part=`. The Lambda looks up the S3 key in DynamoDB (the client never supplies a key), signs a five-minute GET URL, and the browser fetches the PDF from the private bucket.
- **Upload**. The administrator copies a PDF named like 1967/1967-March-1.pdf into the bucket. S3 notifies the indexer Lambda, which parses the name and writes (or, on delete, removes) the category entry. No manual step keeps the catalog in sync.

Why each service, and the alternative I rejected

Choice	Rejected alternative	Why
S3 for PDFs and the site	EBS or EFS	PDFs are static, unstructured objects that need cheap, durable, HTTPS-native retrieval. EBS and EFS are block/file storage for a running server, which would force an always-on instance.
DynamoDB for the catalog	RDS	The catalog is a key lookup (year, month, part) with no joins. On-demand DynamoDB has no idle instance cost and no schema migrations; an RDS instance would bill while idle.
Lambda for all backend logic	EC2 or ECS Fargate	Traffic is tiny and bursty, so paying per invocation beats a VM or container running 24/7. Fargate is normally fronted by a load balancer, which bills hourly.
API Gateway HTTP API	REST API; Application Load Balancer	Lower per-request price and latency than a REST API, and I need none of its extra features. An ALB charges by the hour even with zero traffic.
CloudFront in front of everything	S3 static website hosting	Gives HTTPS, edge caching and one domain for site and API (no CORS), and lets the website bucket stay private through Origin Access Control. S3 website hosting requires a public bucket and is HTTP-only.

Pre-signed URLs	Public bucket; streaming PDFs through Lambda	A public bucket exposes every journal permanently. Proxying through Lambda adds payload limits and pays compute for transfer time. Signed URLs keep the bucket private and expire in five minutes.
S3 event to indexer Lambda	Manual DB entries; scheduled scan	Event-driven, so the catalog updates within seconds of an upload with no polling and no human step.
One IAM role per Lambda	One shared role; access keys	Each function gets only the actions it needs, and there are no long-lived keys to leak.
CloudFormation	Terraform; console clicks	Native to AWS with nothing extra to install, one template for the whole stack, and a clean delete. Console clicks are not repeatable or reviewable.

Implementation

Steps to reproduce

1. In CloudFormation, create a stack from template.yaml (stack name journal-archive, in lowercase; set the OwnerName parameter; acknowledge that IAM roles will be created). Creation takes roughly 5-10minutes, mostly CloudFront.
2. Read the stack Outputs: the CloudFront site URL and the names of the website and PDF buckets.
3. Upload index.html to the website bucket.
4. Upload the year folders of PDFs to the PDF bucket. The indexer fills DynamoDB automatically.
5. Open the CloudFront URL, choose a year, and download a part.

Key configuration

- Both buckets block all public access and use server-side encryption with Amazon S3-managed keys (SSE-S3) using AES-256 encryption and the PDF bucket is versioned. The website bucket is readable only by this CloudFront distribution (Origin Access Control).
- One distribution, two origins. The default behaviour caches the static site; the /api/* behaviour uses a no-cache policy (so signed URLs are never shared between visitors) and forwards the query string.
- DynamoDB: partition key *year*, sort key *MM-part* so months sort in order; on-demand billing and point-in-time recovery on.
- Three Python 3.12 Lambdas, each with its own least-privilege role. API Gateway throttles each route to 10 requests/second (burst 20). The S3 trigger fires on created and removed objects ending in .pdf.

Challenges and Solutions

- First deployment rolled back. CloudFront rejected the distribution with “the specified cache policy does not exist,” because I had hard-coded the IDs of CloudFront’s built-in policies. I deleted the failed stack (since a rolled-back stack cannot be reused) and changed the template to define its own cache and origin-request policies, which removed every hard-coded ID.
- Downloads failed with a region error. S3 answered “the region ‘us-west-2’ is wrong, expecting ‘us-east-1.” The default Lambda S3 client signed the URL against the global S3 endpoint, which expect us-east-1 for a request scoped to us-west-2, so S3 refused it. I configured the client with the regional endpoint and applied a direct stack update; no data had to be re-uploaded.
- Misleading signals. The Events tab keeps old IN_PROGRESS rows after a resource finishes, so the stack status is the source of the truth. Opening an objects S3 URL gives Access Denied and that is expected, because the side is reachable only through the CloudFront domain.

How I tested

- The site loads from the CloudFront domain.
- Uploading 1967-March-1.pdf produces an indexed line in the indexer’s CloudWatch log, a row in DynamoDB, and a Part 1 button on the site within seconds.
- Clicking the button downloads the PDF.
- A misnamed or missing archive file produces a Skipping unrecognized key log line and does not appear.
- The direct S3 URL returns Access Denied, confirming the bucket is private.
- To test it from outside my own network and account, I gave the CloudFront domain to a family member in Canada, who needed no AWS account or login. They were able to access the site and download journals successfully. In this case, the browser called the /api/journals endpoint and API Gateway invoked the list Lambda, which scanned the DynamoDB in us-west-2. When they clicked a part, the download Lambda looked up the file in DynamoDB and signed a five-minute URL, and their browser then fetched the PDF directly from the private S3 bucket. This confirmed that the site works for an outside user in another country with no credentials of mine, and that the pre-signed URL is the only way to reach the PDF, which is exactly it’s intended purpose.

Architectural Concerns

Security

No access keys exist anywhere; each Lambda assumes its own role (list: dynamodb:Scan; download: dynamodb:Query plus s3:GetObject on the PDF bucket only; indexer: PutItem and DeleteItem). Neither bucket is public, and PDFs are reachable only through URLs that expire after five minutes. Data is encrypted at rest (S3 SSE and DynamoDB default encryption) and in transit (CloudFront redirects HTTP to HTTPS, and API origin is HTTPs-only, signed URLs are

HTTPS). The download API accepts year, month and part, never an S3 key, so a caller cannot request an arbitrary object. There are no security groups because there are no servers.

Reliability

S3, DynamoDB, Lambda, API Gateway, and CloudFront are managed services that run across multiple Availability Zones, so an AZ failure needs no action from me. The PDF bucket is versioned (recovers overwrites and deletes), and DynamoDB has point-in-time recovery. The catalog is derived data, and it can be rebuilt by re-triggering the indexer over the bucket. **RPO**: effectively zero for the PDFs already in S3 and seconds for the catalog. **RTO**: minutes for a deleted catalog or a broken stack (redeploy the template, re-sync the site file). The DR strategy is backup and restore within one region. A full Region outage would mean redeploying elsewhere and re-uploading from my original local copies so that RTO is hours and depends on upload speed.

Performance

The static site is cached at CloudFront edge locations and the PDFs come directly from S3. Lambda and on-demand DynamoDB scale with request volume without configuration. The bottleneck is the list endpoint, which performs a DynamoDB scan of the whole table on every page load which is fine at a few hundred items, but cost and latency grow with the table. The fix is a per-year Query or caching the response at CloudFront. The API throttle (10 requests/second per route) is also a deliberate ceiling that protects cost but would limit a sudden traffic spike.

Cost

Nothing in the design bills while idle, there is no NAT gateway, load balancer or always-on compute. Estimate from AWS: $0.09 per month, with no upfront cost and $1.08 over 12 months (AWS Pricing Calculator, US West (Oregon): (https://calculator.aws/#/estimate?id=2b65b03c64828cdf579c33c9a1dc23d7c8069a2f)). Assumptions: 624 PDFs, two parts per month, averaging 5 MB or about 3 GB and 1,000 visits per month, each loading the site, listing the journals once and downloading three PDFs (3,000 downloads, about 15GB). Line items from the calculator: S3 $0.07, CloudFront $0.02, and API Gateway, Lambda and DynamoDB each $0.00 after rounding (API Gateway is about $0.004 for 4,000 HTTP API calls; DynamoDB is on-demand). S3 data transfer out is $0, because 15 GB is under the 100 GB monthly free allowance. Optimizations applied: an HTTP API instead of a REST API, and DynamoDB on-demand billing with CloudFront price class 100. Would apply: an S3 lifecycle rule moving older years to Standard-IA, and caching the journal list at CloudFront to cut Lambda and DynamoDB reads.

Well-Architected Self Review

Pillar	One thing the design does well	One thing I would improve
Security	Private buckets, Origin Access Control, least-privilege role per function, and short-lived signed URLs mean no data is exposed by default.	The API is public with no authentication, and there is no WAF or per-client rate limit beyond API Gateway's per-route throttle. Anyone can request download links as fast as the throttle allows.
Reliability	Fully managed multi-AZ services plus versioning and point-in-time recovery; the catalog can be rebuilt from S3.	Single region with no cross-region replication, and no CloudWatch alarms on Lambda errors. I have not rehearsed recovery beyond redeploying the stack.
Cost Optimization	Everything is pay-per-use, so an idle archive costs almost nothing.	No S3 lifecycle tiering for older journals, and the uncached list endpoint scans the table on every visit.

What I would do differently

I would deploy a minimal version of the stack on day one and test each path (browse, index, download) before polishing, because both of my deployment bugs were runtime assumptions. I could add authentication in front of the API (Amazon Cognito), switch the list call to a per-year query behind a CloudFront cache, add CloudWatch alarms on Lambda errors, move old years to cheaper S3 storage, and replicate the PDF bucket to a second region. After finalizing everything, I tore the deployment down by emptying both buckets, deleting the stack, and removing the Lambda log groups.